



NileRoute OS

≡ Can show publicly	Yes
≡ Category	Logistics & Trade
🌐 Country	France Morocco Qatar Saudi Arabia Sudan
🔗 Industry	Logistics, trade, route intelligence, operations software
≡ Main features	- - -
🔗 Project link	https://nileroute-os.vercel.app/
≡ Project type	Client Work / Internal Product / Productized Prototype /Demo
≡ Proof available	Public link: Screenshots: Demo: Testimonials: Metrics:
≡ Status	Live
≡ Technical stack	- - -

I'll audit this repository systematically to produce a factual portfolio report. Let me start by exploring the project structure. Now let me explore the repository structure and examine key files: # Project Repository Report

1. Project identity

- Project name: NileRoute OS
- Repository name: khairreddine-arbouch/nileroute-os
- Project category: Logistics & Freight Operations Management
- Apparent project type: Internal product / SaaS dashboard
- Public-facing or internal-facing: Internal-facing (operations team dashboard)
- Main user/customer: Freight logistics dispatchers, operations managers, document controllers, sales teams

2. One-sentence factual description

NileRoute OS is a multi-tenant, dark-themed freight logistics operations dashboard built with Next.js and Supabase that manages customers, shipment requests, active shipments, containers, tracking events, documents/drive, and includes an AI assistant for operational queries.

3. What the project does

- **Customer management:** Create, organize, and track customers with profiles including contacts, documents, operations history, and activity timelines; supports import/export and saved views.
- **Shipment lifecycle:** Full pipeline from intake (requests) → quoting → creation → execution, with status tracking across 12 statuses (New Request → Delivered), risk assessment, revenue tracking, and document checklists.
- **Real-time tracking:** Event-based tracking feed capturing operational signals (ETAs, status changes, gate events) with optional view filters by shipment.
- **Container management:** Track containers linked to shipments with volume/weight utilization, vessel assignments, and seal numbers.
- **Documents & drive:** Workspace-wide file storage with folder organization, upload/download, document lifecycle (pending → approved → rejected), compliance tracking, and storage quota management.
- **AI assistant:** Context-aware chat workspace (persisted per workspace) powered by NVIDIA LLM inference; generates operational summaries, risk analysis, and document status reports with citations to records and real data.

4. Confirmed implemented features

Feature	What it does	Evidence file paths	Confidence: High / Medium / Low
Multi-tenant authentication	Clerk-based sign-in/up with workspace/organization separation	<code>app/layout.tsx</code> , <code>.env.example</code> , <code>lib/auth/profile.ts</code>	High
Customer CRUD	Create, read, update customers with full profile (contacts, documents, operations tab)	<code>app/(ops)/customers/page.tsx</code> , <code>features/customers/actions.ts</code> , <code>supabase/migrations/001_*.sql</code>	High
Shipment management	Create shipments, assign status from predefined pipeline, track revenue/margin/risk	<code>app/(ops)/shipments/page.tsx</code> , <code>features/shipments/types.ts</code> , <code>features/shipments/queries.ts</code>	High
Shipment requests (intake)	Create requests with customer/container/route info, list and triage requests	<code>app/(ops)/requests/page.tsx</code> , <code>features/requests/options.ts</code> , <code>supabase/migrations/001_*.sql</code>	High
Container tracking	List containers, link to shipments, track volume/weight utilization	<code>app/(ops)/containers/[id]/page.tsx</code> , <code>features/containers/queries.ts</code>	High
Dashboard KPIs	Real-time aggregation of shipments in transit, delivered, high-risk; revenue pipeline, top customers, missing docs, open requests	<code>app/(ops)/dashboard/page.tsx</code> (lines 24–102)	High
Tracking events	Append-only event feed with workspace-scoped visibility	<code>app/(ops)/tracking/page.tsx</code> , <code>supabase/migrations/005_*.sql</code> (activity_events immutable trigger)	High
Document/drive storage	Upload files to workspace with folder hierarchy, metadata (file size, MIME, upload date, uploader)	<code>features/drive/actions.ts</code> , <code>features/drive/queries.ts</code> , <code>supabase/migrations/009_*.sql</code>	High
S3/Supabase storage abstraction	Support for S3-compatible backends (AWS S3, Cloudflare R2, MinIO) with signed URL download	<code>features/drive/actions.ts</code> , <code>.env.example</code> (S3_* vars)	High
Document lifecycle	Upload status tracking (pending → active → deleted) separate from review status (pending → approved → rejected)	<code>supabase/migrations/009_*.sql</code>	High
AI assistant workspace	Persistent chats and folders per workspace; SSE-based streaming responses	<code>app/(ops)/assistant/page.tsx</code> , <code>app/api/ai/chat/route.ts</code> , <code>features/ai/components/assistant-workspace.tsx</code>	High

Feature	What it does	Evidence file paths	Confidence: High / Medium / Low
AI inference via NVIDIA	Configurable LLM provider (NVIDIA API with llama-3.1-70b-instruct default); rate limiting per user/workspace	<code>features/ai/nvidia-client.ts</code> , <code>features/ai/rate-limit.ts</code>	High
AI context tools	Structured prompts include shipment, customer, request, tracking, document, and financial summaries with citations	<code>app/api/ai/chat/route.ts</code> (system prompt), <code>features/ai/tools.ts</code>	Medium
Role-based access control (RBAC)	Seven roles (owner, admin, manager, operations, sales, finance, viewer) with granular action permissions	<code>lib/auth/permissions.ts</code> (ROLE_PERMISSIONS map with 66 actions)	High
Workspace/org isolation	RLS policies enforcing workspace-scoped and org-scoped data access; separate service-role for server actions	<code>supabase/migrations/008_*.sql</code> , <code>supabase/migrations/010_*.sql</code>	High
Saved table views	User-scoped named filters/sorts/column preferences per entity type	<code>supabase/migrations/013_*.sql</code> , <code>features/saved-views/queries.ts</code>	High
Activity audit trail	Append-only activity_events table with constraints preventing mutation/deletion	<code>supabase/migrations/005_*.sql</code>	High
Member invitations	Invite pending users with hashed tokens; accept via <code>/invite/accept?token=...</code>	<code>supabase/migrations/008_*.sql</code> (invitations table, tokens)	Medium

5. Partially implemented or unclear features

Feature / claim	Why it is unclear or partial	Evidence file paths	What needs confirmation
AI risk assessment on shipments	Dashboard shows <code>aiRisk</code> field (low/medium/high) on shipments; unclear how/when risk is computed (real AI inference vs. static field)	<code>app/(ops)/dashboard/page.tsx</code> (line 35: <code>shipment.aiRisk</code>), <code>features/shipments/types.ts</code> (ShipmentRisk type)	Is <code>aiRisk</code> populated by AI inference, user input, or external API?
AI tool citations and charts	Assistant UI references <code>AiToolCitation</code> and <code>AiChartSpec</code> types; unclear if charts are generated server-side or if	<code>features/ai/types.ts</code> (<code>AiToolCitation</code> , <code>AiChartSpec</code>), <code>app/api/ai/chat/route.ts</code>	Do charts render in practice? Are citations always populated?

Feature / claim	Why it is unclear or partial	Evidence file paths	What needs confirmation
	citations are always available		
Document compliance/requirements	<code>document_requirements</code> table exists for shipment/request/customer document checklists; unclear if UI fully implements checklist UI or just data model	<code>supabase/migrations/015_*.sql</code>	Is the document checklist UI complete in the frontend?
Workflow automation / AI workflows	Permission model includes <code>ai:workflow:read</code> , <code>ai:workflow:write</code> , <code>ai:action:approve</code> ; no visible workflow UI in routes	<code>lib/auth/permissions.ts</code> (lines 36–38), <code>features/ai/types.ts</code>	Are workflows implemented in UI? Is this a placeholder for future work?
AI search / semantic queries	No evidence of search/semantic indexing (MeiliSearch, Elasticsearch) in code; <code>.env.example</code> has commented-out <code>MEILISEARCH_*</code> vars	<code>.env.example</code> (lines 125–127)	Is semantic search planned or disabled?
PDF rendering / document generation	<code>@sparticuz/chromium-min</code> and <code>puppeteer-core</code> in dependencies suggest PDF generation capability; no visible route/action implements it	<code>package.json</code> (lines 52, 66), <code>.env.example</code> (lines 109–115: <code>CHROMIUM_PACK_URL</code>)	Is PDF export/generation implemented? Are there hidden routes?
Email notifications	<code>.env.example</code> references optional <code>RESEND_API_KEY</code> and <code>EMAIL_FROM</code> ; no email sending code visible	<code>.env.example</code> (lines 131–133)	Is email enabled in production? Any email workflows?
Background jobs (Inngest / Trigger.dev)	<code>.env.example</code> has placeholders for <code>INNGEST_*</code> ; no actual job queue implementation visible	<code>.env.example</code> (lines 119–121)	Are background jobs used, or is this for future work?
Financial data (costs, margins, payment status)	Shipments track <code>revenue</code> , <code>margin</code> , <code>paymentStatus</code> , <code>costAmount</code> ; unclear if these are user-entered or calculated	<code>features/shipments/types.ts</code> (lines 103–106), <code>app/(ops)/dashboard/page.tsx</code> (line 36–39)	Are costs/margins auto-calculated or manual entry?

6. Mocked, hardcoded, or demo-only parts

Area	What appears mocked/demo-only	Evidence file paths	Risk if claimed publicly
Customer seed data	Three hardcoded demo customers (Nile Bridge Trading, Al Noor Trading, Khartoum Steel) with Sudanese addresses/phone numbers; feature flag <code>USE MOCK OPS</code> controls fallback	<code>features/customers/data.ts</code> (lines 1–82), <code>.env.example</code> (line 69)	Claiming real customer data when only demo data is in repo; risk of confusing prospect about real vs. test customers.
Drive/documents mock data	<code>MOCK_BUNDLE</code> with hardcoded folder/file entries (contracts, invoices, shipment templates, ~5.3 GB storage) used when <code>ALLOW MOCK DRIVE FALLBACK=true</code>	<code>features/drive/data.ts</code> (MOCK_FOLDERS, MOCK_FILES, MOCK_BUNDLE), <code>features/drive/queries.ts</code> (allowMockDriveFallback flag)	If mock fallback is enabled in production, demo files may appear as real data; misleading for live demo.
Workspace users hardcoded	<code>WORKSPACE_USERS</code> list (Ahmet Yılmaz, Fatma Kaya, Mustafa Öztürk) used as placeholder team members	<code>features/customers/constants.ts</code> (lines 1–4)	Static team names; real workspace members should come from database.
Seed SQL for demo data	<code>supabase/seed/demo_data.sql</code> provides bulk insert of demo customers, requests, shipments, containers, tracking events, documents, chats	<code>supabase/seed/README.md</code> , <code>supabase/seed/demo_data.sql</code>	Explicitly documented as demo; not a hidden risk if clearly labeled in setup.
Storage file paths	<code>file_path</code> / <code>storage_key</code> values are placeholders; signed URLs won't resolve to real objects unless matching keys uploaded to bucket	<code>supabase/seed/README.md</code> (line 30)	Claiming full file storage when bucket keys don't match; will show broken download links.
Tracking events	Dashboard/tracking queries use real seed data if seeded; no indication if production instances have live tracking feeds or are static	<code>supabase/seed/demo_data.sql</code> (tracking_events inserts)	Unclear whether tracking is live operational data or demo snapshot.

7. Technical stack

Layer	Technologies confirmed	Evidence file paths
Frontend framework	Next.js 16.2.6 (App Router), React 19, TypeScript 5.7.3	<code>package.json</code> (lines 64, 68, 88)
UI framework	Tailwind CSS 4.2.0, Radix UI components (15 packages), shadcn-style custom components	<code>package.json</code> (lines 79, 26–50), <code>components/</code> directory
Forms & validation	React Hook Form 7.54.1, Zod 3.24.1	<code>package.json</code> (lines 71, 76)
Charts & visualization	Recharts 2.15.0, date-fns 4.1.0	<code>package.json</code> (lines 73, 59)
Animation	Framer Motion 12.38.0, embla-carousel-react 8.6.0	<code>package.json</code> (lines 61, 60)
Command palette	cmdk 1.1.1	<code>package.json</code> (line 58)
Auth	Clerk (@clerk/nextjs 7.3.5, @clerk/ui 1.11.0)	<code>package.json</code> (lines 20, 21), <code>.env.example</code> (lines 29–47)
Database	Supabase (PostgreSQL + RLS), @supabase/supabase-js 2.105.4	<code>package.json</code> (line 53), <code>.env.example</code> (lines 50–60), <code>supabase/migrations/</code>
Object storage	S3-compatible (@aws-sdk/client-s3 3.1053.0, @aws-sdk/s3-request-presigner 3.1053.0); Supabase Storage or AWS S3/Cloudflare R2	<code>package.json</code> (lines 18, 19), <code>.env.example</code> (lines 74–106)
PDF rendering	@sparticuz/chromium-min 149.0.0, puppeteer-core 25.1.0	<code>package.json</code> (lines 52, 66), <code>.env.example</code> (lines 109–115)
Drag & drop	@dnd-kit (core, sortable, utilities)	<code>package.json</code> (lines 22–24)
AI/LLM	NVIDIA API client (proprietary integration); llama-3.1-70b-instruct model (configurable)	<code>features/ai/nvidia-client.ts</code> , <code>app/api/ai/chat/route.ts</code>
Rate limiting	In-memory rate limiter (per-user, per-workspace buckets)	<code>features/ai/rate-limit.ts</code>
Package manager	pnpm (pnpm-lock.yaml)	<code>package.json</code> (workspaces reference), setup scripts
Deployment	Vercel (analytics via @vercel/analytics 1.6.1); environment assumes serverless Next.js	<code>package.json</code> (line 54), <code>.env.example</code> (lines 11–23)
Testing	Node test runner (native), TypeScript @types packages	<code>package.json</code> (line 10: <code>node --test</code>)
Linting	ESLint 9.39.4, ESLint config for Next.js	<code>package.json</code> (lines 9, 84)
Design tokens	PostCSS 8.5, Tailwind CSS 4.2.0; design system maintained in <code>DESIGN.md</code>	<code>package.json</code> (lines 85, 79), README references <code>../DESIGN.md</code>
Observability	Error reporting (reportError calls throughout), Sentry optional	<code>.env.example</code> (lines 137–142: <code>SENTRY_*</code>), <code>lib/observability/report.ts</code>
Analytics	@vercel/analytics (optional, gated on NODE_ENV)	<code>package.json</code> (line 54), <code>app/layout.tsx</code>

Layer	Technologies confirmed	Evidence file paths
Not found in repo	GraphQL, tRPC, background job queue (Inngest placeholders only), email service (Resend optional), search (MeiliSearch optional), maps/geospatial	<code>.env.example</code> references but not implemented

8. Routes, pages, and user flows

Route/Page	Purpose	Evidence file paths
<code>/sign-in</code> , <code>/sign-up</code>	Clerk authentication UI	<code>README.md</code> (line 73), <code>.env.example</code> (lines 41–47)
<code>/invite/accept?token=...</code>	Accept workspace invitation	<code>README.md</code> (line 74)
<code>/</code>	Root redirect to <code>/dashboard</code>	<code>README.md</code> (line 72)
<code>/dashboard</code>	Command center: KPIs (shipments in transit, delivered, high-risk, revenue), status breakdown chart, risk breakdown, recent 5 shipments, top 5 customers, missing docs/open requests alerts, quick action links	<code>app/(ops)/dashboard/page.tsx</code> , <code>README.md</code> (line 89)
<code>/customers</code>	Customer list (table/kanban toggle), filters, bulk import/export, saved views, create/edit inline or via sheet	<code>app/(ops)/customers/page.tsx</code> , <code>README.md</code> (line 92)
<code>/customers/[id]</code>	Customer profile: General, Contacts, Documents, Operations (shipments for this customer), Activity tabs; breadcrumb navigation	<code>README.md</code> (line 93)
<code>/shipments</code>	Shipment list (table/kanban toggle), filters by status/risk/customer, create, assign owner, bulk actions, CSV export	<code>README.md</code> (line 94), <code>app/(ops)/shipments/page.tsx</code>
<code>/shipments/[id]</code>	Shipment detail: status, route, vessel/ETA, documents checklist, operational risk/timeline narrative, revenue, containers	<code>README.md</code> (line 95)
<code>/requests</code>	Shipment requests intake pipeline; list, create, triage, assign owner	<code>app/(ops)/requests/page.tsx</code> , <code>README.md</code> (line 96)
<code>/requests/[id]</code>	Single request detail with nested customer/container/route data	<code>README.md</code> (line 97)
<code>/tracking</code>	Tracking events feed (real-time signals: gate in/out, status changes, ETAs); optional <code>?view=shipment</code> lens	<code>app/(ops)/tracking/page.tsx</code> , <code>README.md</code> (line 98)
<code>/containers</code>	Container list; links to parent shipment when bound	<code>README.md</code> (line 99), <code>app/(ops)/containers/[id]/page.tsx</code>
<code>/documents</code>	Documents & drive browser with folder tree, file uploads/downloads, rename, move, delete; <code>/drive</code> redirects here	<code>app/(ops)/drive/page.tsx</code> (redirect), <code>README.md</code> (line 100)

Route/Page	Purpose	Evidence file paths
<code>/drive</code>	Alias redirect to <code>/documents</code>	<code>app/(ops)/drive/page.tsx</code>
<code>/assistant</code>	AI chat workspace (persistent chats, folders, SSE streaming); <code>/command</code> redirects here	<code>app/(ops)/assistant/page.tsx</code> , <code>README.md</code> (line 91)
<code>/command</code>	Legacy alias redirect to <code>/assistant</code>	<code>app/(ops)/command/page.tsx</code>
<code>/settings</code>	Workspace settings: members, invitations, teams, roles, activity; admin-only	<code>README.md</code> (line 101)
Shared shell	All <code>/ops/*</code> routes share <code>app/(ops)/layout.tsx</code> : left sidebar (nav), top header (breadcrumbs, workspace switcher, ⌘K hint), command palette, notifications, banners	<code>app/(ops)/layout.tsx</code> , <code>README.md</code> (section "The operations shell")

9. Data model and backend logic

Database entities (from migrations):

- `workspaces` — multi-tenant partitions with `org_id`, `timezone`, storage quota
- `organizations` — company/tenant accounts; plan tier; `clerk_org_id` for webhook sync
- `profiles` — users with Clerk ID, full name, email, avatar
- `workspace_members` — user ↔ workspace membership with role (owner, admin, manager, operations, sales, finance, viewer) and status
- `organization_members` — user ↔ org membership (for cross-workspace permissions)
- `customers` — company/individual accounts with address, contact, currency, credit limit, status (active/lead/inactive/blocked), assigned owner
- `contacts` — people linked to customers (role, email, phone, preferred channel)
- `shipment_requests` — intake records: customer, origin/destination, cargo type, container count, status, priority
- `shipments` — operational shipments with status (12-step pipeline), revenue/cost/margin, aiRisk (low/medium/high), vessel/voyage, ETA, documents
- `containers` — physical units linked to shipments; volume/weight tracking, seal numbers, status
- `tracking_events` — append-only operational signals (immutable by trigger)
- `documents` — file metadata; workspace/customer/shipment scoped; `upload_status` (pending → active → deleted), `review_status` (pending → approved)
- `drive_folders` , `drive_files` — hierarchical workspace drive; soft delete support
- `ai_chats` , `ai_chat_messages` — persistent AI assistant per workspace
- `activity_events` — append-only audit log (mutation-prevented by trigger)

- `document_requirements` — checklist items for shipments/requests/customers
- `user_table_views` — saved filters/sorts per user, workspace, entity type
- `invitations` — pending team member invites with hashed tokens

API endpoints:

- `POST /api/ai/chat` — stream LLM response with context (SSE); rate-limited
- `GET/POST /api/ai/chats` — list/create chats
- `GET/PATCH/DELETE /api/ai/chats/[id]` — chat CRUD
- `GET/POST /api/ai/folders` — list/create folders
- `GET/PATCH/DELETE /api/ai/folders/[id]` — folder CRUD
- Webhook routes: `/api/webhooks/clerk/route.ts` (org/user sync)

Server actions (Next.js form/mutation):

- Customer CRUD (`features/customers/actions.ts`)
- Shipment mutations (`features/shipments/actions.ts`)
- Drive file upload/delete/rename/move (`features/drive/actions.ts`)
- Request CRUD (`features/requests/actions.ts`)
- AI chat persistence (`features/ai/persistence.ts`)

Business logic:

- Multi-tenancy: Workspace scoping + organization hierarchy; RLS policies enforce isolation
- Permission model: Seven roles with action-based RBAC (`lib/auth/permissions.ts`)
- AI context: System prompt instructs LLM to cite records only from workspace data; supports customer, shipment, request, container, document, tracking, finance context types
- Rate limiting: Per-user and per-workspace token buckets (5 min window by default)
- Document lifecycle: Separate `upload_status` (storage state) and `review_status` (compliance state)
- Activity immutability: Trigger prevents updates/deletes on `activity_events`

Data sources:

- Real: User-entered customer/shipment/request data in Supabase (RLS-scoped per workspace/org)
- Seeded demo: `supabase/seed/demo_data.sql` populates demo workspace with customers, requests, shipments, containers, tracking events
- Static: Hardcoded seed customer/file data (`features/customers/data.ts` , `features/drive/data.ts`) as fallback when DB unavailable

10. AI-specific analysis

- **AI provider/model:** NVIDIA API (configurable endpoint via `NVIDIA_BASE_URL`); default model is `meta/llama-3.1-70b-instruct` (configurable via `NVIDIA_MODEL` or `AI_MODEL` env var)
- **AI features implemented:**
 - **Operational Q&A chat:** Users ask questions about shipments, customers, requests, tracking, documents, finance; LLM responds with workspace-scoped data summaries.
 - **Context-aware:** System prompt includes operational context built from shipments, customers, requests, containers, tracking, documents, and financial data; structured as numbered reference blocks so LLM can cite them.
 - **Citations:** Response includes `AiToolCitation` objects linking answers back to specific records (customer, shipment, request, container, document, dashboard, finance) with entity type, href path, label, and optional quote/page.
 - **Charts (transient):** Server can emit `AiChartSpec` (bar/line/pie) as SSE events before LLM streaming; rendered inline in UI but not persisted.
 - **Rate limiting:** Per-user (default 20/min) and per-workspace (default 120/min) limits; configurable via `AI_RATE_LIMIT_*` env vars.
 - **Persistence:** Chats and messages saved to `ai_chats` / `ai_chat_messages` tables; organization/workspace/user scoped; archived/deleted support.
 - **Workflows (partial):** Permission model includes workflow read/write/approve actions, but no UI or implementation visible; appears to be placeholder for future work.
- **Prompts:** System prompt defined in `app/api/ai/chat/route.ts` (lines ~36–62); instructs model to cite from context, avoid fabrication, use workspace codes/names, rank by urgency, link markdown references only to provided paths.
- **Output handling:**
 - Chat messages saved to database after user submission.
 - LLM responses streamed via SSE to client; text, citations, activity steps, and chat ID sent as separate events.
 - Charts emitted as transient events (not persisted with message).
 - Citations stored as `AiToolCitation[]` in message row.
- **Evidence of real inference:** Rate-limit enforcement, contextual prompt construction on each request, LLM configuration (temperature, top_p, max_tokens), streaming response pipeline—all point to real API calls, not static responses.

11. Portfolio table improvements

Field	Recommended value
Status	Alpha / Early beta (8 days old per repo created_time; core CRUD ops solid, AI assistant functional, but gaps in workflows, PDF generation, background jobs)
Industry	Logistics & Freight Operations

Field	Recommended value
Category	SaaS Operations Dashboard
Project type	Internal product (built on behalf of a customer or internal team)
What we built	Multi-tenant freight operations platform with customer/shipment/request/container management, real-time tracking, document lifecycle, workspace-level file storage, and AI-powered operational copilot (NVIDIA LLM with workspace-scoped context).
Client/user problem	Freight dispatchers and operations teams need a unified, dark-themed "operations deck" to manage customer relationships, shipment pipelines, real-time tracking signals, compliance documents, and quickly answer operational questions (ETA misses, risk, missing docs).
Main features	12-step shipment pipeline, role-based access (7 roles), AI chat with citations and risk analysis, document compliance checklist, real-time tracking feed, workspace/org isolation, custom saved views, activity audit trail.
Technical stack	Next.js 16 + React 19, Supabase (PostgreSQL + RLS), Clerk auth, NVIDIA LLM, Tailwind + Radix UI, S3-compatible storage, Vercel deployment.
Proof available	✓ Code review (features folder, migrations, routes fully visible); ✓ Type definitions and schema (Supabase migrations show data model); ✗ Screenshots (none in repo); ✗ Live demo URL (homepage in meta is https://nileroute-os.vercel.app but access not confirmed); ✗ User testimonials; ✗ Production metrics.
Best sales angle	"Complete freight ops dashboard in one dark interface: manage pipeline, track shipments live, automate document compliance, and ask your AI copilot anything about your workspace—all multi-tenant, secure, and permissions-aware."
What should not be claimed	✗ "Production-ready" (repo created 8 days ago; too new); ✗ "Real customer data in demo" (only seed data, with Sudanese demo customers hardcoded); ✗ "Live tracking feeds" (unclear if production instances have real or static tracking); ✗ "PDF generation" (chromium libs present but no visible UI/route); ✗ "Integrated email/background jobs" (env placeholders only); ✗ "Semantic search" (MeiliSearch commented out); ✗ "Production revenue impact" (no metrics, testimonials, or client case study).

12. Safe public claims

- **Next.js + React 19** — Latest frontend stack with App Router, clearly visible in package.json and routes.
- **Supabase + PostgreSQL** — Multi-tenant database with RLS, confirmed in `.env.example` and 18 migration files.
- **Clerk authentication** — Third-party auth with org/workspace webhook integration, clearly implemented.
- **Seven-role RBAC** — Owner, admin, manager, operations, sales, finance, viewer with 66 granular actions; fully modeled in `lib/auth/permissions.ts`.
- **Shipment pipeline** — 12 defined statuses (New Request → Delivered → Closed); visible in type definitions and dashboard aggregation.
- **AI assistant with context** — NVIDIA LLM-powered chat per workspace; context includes shipment, customer, request, tracking, document, and financial summaries; rate-limited and

persisted.

- **Document lifecycle** — Upload status (pending → active → deleted) separate from review status (pending → approved → rejected); implemented in migrations.
- **Real-time tracking** — Append-only tracking_events table with trigger protection; visible in queries and dashboard.
- **S3-compatible storage** — AWS SDK integration with signed URLs; environment variables support AWS S3, Cloudflare R2, MinIO, GCS.
- **Workspace isolation** — RLS policies, organization_id on all operational tables, service-role vs. anon key separation; enforced at schema layer.
- **Saved table views** — User-scoped named filters/sorts per entity (customer, shipment, etc.); implemented in `user_table_views` table and persistence layer.
- **Activity audit trail** — Immutable append-only activity_events; mutation prevented by database trigger.

13. Unsafe or unproven claims

- **"Production-ready"** — Repo created 8 days ago; lacks production monitoring, load testing, multi-region deployment proof, or SLA documentation.
- **"Live shipment tracking from customers"** — No evidence of customer portal or real-time data ingestion from freight partners; only internal workspace events.
- **"Automated document compliance"** — Document requirements checklist exists; UI implementation unclear; no workflow automation rules visible.
- **"AI risk scoring"** — Shipments have `aiRisk` field; unclear how/when it's populated; may be static or user-entered, not AI-computed.
- **"Handles international shipping workflows"** — Supports country/port fields; no evidence of multi-currency, duty/tariff calculation, or HS code handling.
- **"Integrated email notifications"** — `.env.example` has `RESEND_API_KEY` placeholder; no email-sending code visible.
- **"Background job processing"** — `INGEST_*` env vars commented out; no visible job queue implementation.
- **"PDF export / document generation"** — Chromium libs present; no UI route or action implements export.
- **"Semantic search"** — MeiliSearch vars are commented in `.env.example`; search not implemented.
- **"Enterprise-grade" / "Used by N customers"** — No production deployment, customer logos, or testimonials in repo; appears to be internal/prototype.

14. Missing evidence needed

- **Screenshots** — Dashboard KPI layout, customer profile, shipment detail, tracking feed, AI assistant UI, settings/members screen.
- **Demo video** — End-to-end workflow: create customer → create shipment → track → ask AI assistant question.
- **Live demo URL** — Repo metadata includes <https://nileroute-os.vercel.app> but access/credentials not confirmed.
- **Client testimonial or case study** — Who is using this? What problem did it solve? Metrics (time saved, error reduction, shipping cost savings)?
- **Production deployment metrics** — Uptime, latency, user count, shipments processed, documents stored.
- **Real customer data sample** — Anonymized export showing realistic shipment counts, revenue ranges, document types, tracking event volume.
- **Architecture diagram** — Clerk → Supabase → NVIDIA LLM flow; storage provider abstraction; workspace isolation pattern.
- **Scalability/performance notes** — RLS query latency, storage quota per workspace, AI rate-limit tuning recommendations.
- **Security audit or compliance checklist** — GDPR data residency, SOC 2 readiness, encryption at rest/in transit.
- **Roadmap or feature release notes** — What's next (workflows, reporting, integrations with freight APIs)?

15. Final confidence score

Confidence: 58 / 100

Rationale:

- **Strengths (+):**
 - Codebase is well-architected (clear separation of concerns, feature folders, type safety); migrations are comprehensive and defensive (RLS, immutability patterns); AI integration is real (NVIDIA API, rate limiting, context building). Suitable for technical deep-dive or architecture case study.
 - Core features (customers, shipments, tracking, documents, RBAC) are fully implemented and non-trivial; not a toy project.
 - Multi-tenancy and security patterns (workspace isolation, org hierarchy, RLS) are production-grade abstractions.
- **Weaknesses (-):**
 - Repository is only 8 days old; unclear if this is a real client project, internal proof-of-concept, or portfolio piece.
 - No production deployment evidence, customer data, metrics, or testimonials.

- Critical features appear incomplete or unclear: AI risk scoring (unclear trigger), workflows (permissions exist but no UI), PDF generation (libs present but no route), background jobs (env vars only).
- Mock/demo data is hardcoded and prominent; demo fallback flag could mask real production issues.
- No live demo URL confirmation; homepage URL in metadata may not be accessible.

Safe to include in portfolio if:

- Positioned as "technical case study" or "architecture proof-of-concept" (not production SaaS).
- Screenshots/demo video added to show actual UI.
- Clarified which features are complete vs. planned (workflows, PDF export, email).
- Client permission and/or testimonial obtained.

Not safe to claim as:

- "Production SaaS used by X customers" (no evidence).
- "Handles enterprise compliance" (features exist but unproven at scale).
- "Live real-time tracking system" (unclear if real data or demo only).